



Instituto Politécnico  
de Viana do Castelo

# LICENCIATURA EM ENGENHARIA INFORMÁTICA

## OPÇÃO II – APRENDIZAGEM ORGANIZACIONAL / SISTEMAS DE SUPORTE À DECISÃO

Plataforma de GPTs com RAG usando tecnologias Open Source  
e dados estruturados e não estruturados

31427 - Filipa Calheiros  
filipacalheiros@ipvc.pt

Jorge Ribeiro  
• jribeiro@estg.ipvc.pt

## ■ Objetivos

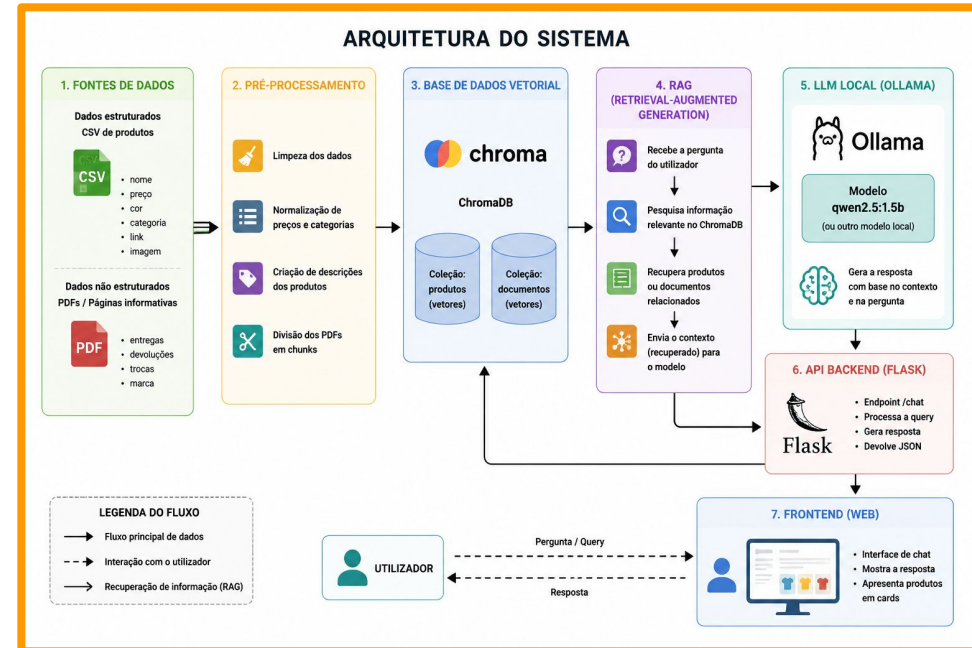
- Desenvolver um assistente inteligente de moda baseado em **RAG (Retrieval-Augmented Generation)**
- Integrar **dados estruturados** (produtos em CSV) para pesquisa e filtragem
- Integrar **dados não estruturados** (PDFs) para responder a questões sobre a loja
- Implementar uma **API em Flask** para processamento de pedidos
- Utilizar **ChromaDB** para armazenamento e pesquisa semântica (embeddings)
- Usar um **LLM local (Ollama)** para geração de respostas
- Criar uma **interface web** simples para interação com o utilizador
- Permitir pesquisa por **preço, cor e tipo de produto**
- Garantir respostas **coerentes e contextualizadas** com os dados disponíveis

## ■ Tema

- O projeto consiste no desenvolvimento de um **assistente inteligente de moda**, capaz de responder a perguntas e sugerir produtos de forma automática
- A solução utiliza **RAG (Retrieval-Augmented Generation)** para combinar pesquisa em base de dados com geração de respostas por um modelo de linguagem
- Foram utilizados **dados estruturados** (produtos extraídos para CSV) para permitir pesquisa por preço, cor e tipo
- Foram também utilizados **dados não estruturados** (conteúdo convertido para PDF) para responder a questões sobre a loja, como entregas e devoluções
- Os dados foram obtidos a partir do site oficial da marca MO (Modalfa):  
<https://mo-online.com/pt/>

## ■ Arquitetura

- Recolha de dados estruturados (CSV de produtos)
- Recolha de dados não estruturados (PDFs informativos)
- Pré-processamento e normalização dos dados
- Armazenamento no ChromaDB (base de dados vetorial)
- Utilização de RAG para pesquisa semântica de contexto
- Processamento da pergunta pelo LLM local (Ollama)
- API Flask para gestão da comunicação
- Frontend web para interação com o utilizador



## ■ Ollama

- Foi utilizado o **Ollama** para executar localmente o modelo de linguagem **qwen2.5:1.5b**
- Inicialmente, o modelo foi descarregado com sucesso através do comando `ollama pull`, garantindo a sua disponibilidade local
- Após a instalação, foi realizado um teste com `ollama run`, confirmando o correto funcionamento do modelo
- O modelo respondeu de forma imediata a uma interação simples, demonstrando capacidade de geração de linguagem natural
- Esta abordagem permite utilizar um **LLM local**, sem necessidade de serviços externos, garantindo maior controlo e privacidade dos dados

```
C:\Users\Asus>ollama pull qwen2.5:1.5b
pulling manifest
pulling 183715c43589: 100%
pulling 66b9ea09bd5b: 100%
pulling eb4402837c78: 100%
pulling 832dd9e00a68: 100%
pulling 377ac4d7aeef: 100%
verifying sha256 digest
writing manifest
success
```

```
C:\Users\Asus>ollama run qwen2.5:1.5b
>>> olá
Olá! Como posso ajudar você hoje? Estou aqui para responder a suas perguntas e auxiliar com qualquer tarefa que precisar.
```

## ■ Dados

- Foram criadas funções para **limpeza e normalização dos dados extraídos** durante o scraping
- A função `clean_price` permite tratar os preços, removendo símbolos e garantindo um formato numérico consistente
- A função `normalize_category` assegura a uniformização das categorias, evitando inconsistências nos dados
- Foi também implementada a função `build_description`, responsável por construir uma **descrição enriquecida do produto**, combinando nome, categoria, subcategoria e cor
- Este processo melhora a qualidade dos dados utilizados no sistema, contribuindo para uma **pesquisa mais precisa e eficaz**

```
def clean_price(price_str):  
    if not price_str or "N/A" in price_str:  
        return ""  
    return price_str.replace("€", "").replace(", ", ".").strip()  
  
def normalize_category(cat):  
    if not cat:  
        return ""  
    cat = cat.strip()  
    if cat.lower() == "shirts":  
        return "T-shirts"  
    return cat  
  
def build_description(name, categoria, subcategoria, color):  
    parts = [name]  
  
    if categoria and categoria.lower() not in name.lower():  
        parts.append(categoria)  
  
    if subcategoria and subcategoria.lower() not in name.lower():  
        parts.append(subcategoria)  
  
    if color and color.lower() not in name.lower():  
        parts.append(f"cor {color}")  
  
    return " ".join(parts).strip()
```

## Dados

- Foi implementado um ciclo para percorrer várias páginas do site da MO, permitindo recolher um conjunto alargado de produtos
- Para cada página, é feito um pedido HTTP e o conteúdo é processado com **BeautifulSoup**
- Os produtos são identificados através dos elementos HTML com a classe product-tile
- Para cada produto, são extraídas informações essenciais como **nome, preço, link e imagem**
- Sempre que disponível, é feito um acesso à página individual do produto para obter dados adicionais como **cor, categoria e subcategoria**
- Estes dados são posteriormente tratados e organizados, permitindo construir uma base estruturada para o sistema de recomendação

```
for page in range(0, 10):
    start = page * 12
    url = f'{base_url}?cgid=823631&start={start}&sz=12'
    print(f'À processar página {page+1}')

    response = requests.get(url, headers=headers)
    soup = BeautifulSoup(response.text, "html.parser")

    products = soup.find_all("div", class_="product-tile")

    if not products:
        break

    for p in products:
        if count >= LIMIT:
            break

        name_tag = p.find("a", class_="link")
        name = name_tag.get_text(strip=True) if name_tag else ""

        link = "https://mo-online.com" + name_tag["href"] if name_tag else ""

        price_tag = p.find("span", class_="value")
        price_raw = price_tag.get_text(strip=True) if price_tag else ""
        price = clean_price(price_raw)

        img_tag = p.find("img")
        image = img_tag["src"] if img_tag else ""

        color = ""
        categoria_principal = ""
        subcategoria = ""

        if link:
            try:
                prod_response = requests.get(link, headers=headers)
                prod_soup = BeautifulSoup(prod_response.text, "html.parser")

                title_tag = prod_soup.find("title")
                if title_tag:
                    title_text = title_tag.get_text()
                    parts = title_text.split(",")
                    if len(parts) >= 3:
                        color = parts[-1].replace(" MO Online", "").strip()

                desc_tag = prod_soup.find("meta", {"name": "description"})
                if desc_tag:
                    descricao_raw = desc_tag.get("content", "")

                if "-" in descricao_raw:
                    categoria_texto = descricao_raw.split("-")[-1]
                    categoria_texto = categoria_texto.replace("MO Online", "").strip()

                if " | " in categoria_texto:
                    partes = categoria_texto.split(" | ")
                    categoria_principal = normalize_category(partes[0])
                    subcategoria = normalize_category(partes[1])
                else:
                    categoria_principal = normalize_category(categoria_texto)
                    subcategoria = ""
```



## ■ Dados

- Após a extração e tratamento da informação, cada produto é organizado numa estrutura de dados
- São armazenados atributos como **nome**, **preço**, **link**, **imagem**, **cor**, **categoria**, **subcategoria** e **descrição**
- A descrição é gerada automaticamente através da função de enriquecimento, melhorando a qualidade dos dados
- Cada produto é adicionado a uma lista final que será posteriormente exportada
- Foi implementado um **contador de produtos** para controlar o número máximo de itens recolhidos
- É também utilizado um pequeno atraso (sleep) entre pedidos para evitar sobrecarga no servidor
- No final, os dados ficam prontos para serem guardados e utilizados no sistema de recomendação

```
except Exception:
    print(f"Erro no produto: {link}")

descricao = build_description(
    name,
    categoria_principal,
    subcategoria,
    color
)

data.append([
    name,
    price,
    link,
    image,
    color,
    categoria_principal,
    subcategoria,
    descricao
])

count += 1
print(f"Produto {count} guardado")

time.sleep(0.3)

if count >= LIMIT:
    break
```



## ■ Dados

- Após a recolha de todos os produtos, é apresentada a quantidade total de dados obtidos
- Os dados são guardados num ficheiro **CSV**, permitindo fácil utilização e integração com outras ferramentas
- São definidas colunas como **Nome, Preço, Link, Imagem, Cor, Categoria, Subcategoria e Descrição**
- Cada produto é armazenado como uma linha no ficheiro
- O ficheiro final (produtos\_mo\_final.csv) constitui a base de dados estruturada utilizada no sistema
- No final do processo, é apresentada uma mensagem a confirmar a criação do CSV com sucesso

```
print(f"Total produtos: {len(data)}")

with open("produtos_mo_final.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f)
    writer.writerow([
        "Nome",
        "Preço",
        "Link",
        "Imagem",
        "Cor",
        "Categoria_Principal",
        "Subcategoria",
        "Descricao"
    ])
    writer.writerows(data)

print("CSV final criado com sucesso!")
```

## ■ Base de Dados

Plataforma de GPTs com RAG usando tecnologias Open Source e dados estruturados e não estruturados

- Foi definida a localização da base de dados utilizando **ChromaDB**, permitindo persistência dos dados
- O ficheiro CSV (produtos\_mo\_final.csv) é utilizado como fonte de **dados estruturados**
- Foram também definidos vários ficheiros PDF contendo informação não estruturada, como **marca, entregas, devoluções e trocas**
- É criado um cliente ChromaDB com armazenamento local
- São configuradas duas coleções distintas:
  - **produtos** → para pesquisa de artigos
  - **documentos** → para informação textual da loja
- Esta separação permite uma melhor organização e utilização dos dados no sistema RAG

```
CHROMA_DATA_PATH = "db"
CSV_FILE = "produtos_mo_final.csv"

PDFS = [
    ("marca.pdf", "marca"),
    ("entregas.pdf", "entregas"),
    ("devolucoes.pdf", "devolucoes"),
    ("trocas.pdf", "trocas")
]

client = chromadb.PersistentClient(path=CHROMA_DATA_PATH)

collection_produtos = client.get_or_create_collection(name="produtos")
collection_documentos = client.get_or_create_collection(name="documentos")
```

## ■ Base de Dados

Plataforma de GPTs com RAG usando tecnologias Open Source e dados estruturados e não estruturados

- Foi implementada uma função para **limpar coleções existentes** na base de dados, evitando duplicação de dados
- A função `extrair_texto_pdf` permite ler o conteúdo dos ficheiros PDF página a página
- O texto extraído é concatenado, formando um bloco contínuo de informação
- Para melhorar a eficiência da pesquisa, o texto é dividido em **pequenos segmentos (chunks)**
- A função `chunk_text` cria estes segmentos com um determinado tamanho e sobreposição, garantindo melhor contexto
- Este processo é essencial para permitir uma **pesquisa semântica mais precisa** dentro dos documentos
- Os chunks resultantes são posteriormente utilizados para gerar embeddings e armazenar no ChromaDB

```
def limpar_colecao(collection):
    try:
        existing = collection.get()
        if existing and existing.get("ids"):
            collection.delete(ids=existing["ids"])
    except Exception:
        pass

def extrair_texto_pdf(path):
    reader = PdfReader(path)
    texto = ""

    for page in reader.pages:
        page_text = page.extract_text()
        if page_text:
            texto += page_text + "\n"

    return texto.strip()

def chunk_text(texto, tamanho=400, overlap=50):
    chunks = []
    start = 0

    while start < len(texto):
        end = start + tamanho
        chunk = texto[start:end].strip()

        if chunk:
            chunks.append(chunk)

        start += tamanho - overlap

    return chunks
```

## ■ Base de Dados

Plataforma de GPTs com RAG usando tecnologias Open Source e dados estruturados e não estruturados

- Foi criada uma função para **carregar e indexar documentos PDF** na base de dados vetorial
- Inicialmente, é feita a limpeza da coleção de documentos para evitar duplicação de dados
- Cada ficheiro PDF é verificado antes de ser processado, garantindo que existe no sistema
- O texto de cada documento é extraído e dividido em **chunks** para melhor desempenho na pesquisa
- Para cada chunk, são associadas **metainformações**, como a origem do ficheiro e o tipo de conteúdo
- Todos os dados são armazenados na coleção **documentos** do ChromaDB
- No final, é apresentada uma mensagem de sucesso ou aviso caso não existam documentos para indexar

```
def carregar_documentos_pdf():  
    limpar_colecao(collection_documentos)  
  
    all_docs = []  
    all_meta = []  
    all_ids = []  
  
    for file_name, tipo in PDFS:  
        if not os.path.exists(file_name):  
            print(f"PDF não encontrado: {file_name}")  
            continue  
  
        texto = extrair_texto_pdf(file_name)  
        chunks = chunk_text(texto)  
  
        for i, chunk in enumerate(chunks):  
            all_docs.append(chunk)  
            all_meta.append({  
                "source": os.path.basename(file_name),  
                "tipo": tipo  
            })  
            all_ids.append(f"{tipo}_{i}")  
  
    if all_docs:  
        collection_documentos.add(  
            documents=all_docs,  
            metadatas=all_meta,  
            ids=all_ids  
        )  
        print("Coleção 'documentos' criada com sucesso!")  
    else:  
        print("Não foram encontrados documentos PDF para indexar.")
```

## ■ Base de Dados

Plataforma de GPTs com RAG usando tecnologias Open Source e dados estruturados e não estruturados

- Foi criada uma função para **carregar os produtos a partir do ficheiro CSV** para a base de dados
- A coleção de produtos é inicialmente limpa para evitar duplicação de dados
- O ficheiro CSV é verificado antes da leitura, garantindo que existe no sistema
- Para cada produto, é construída uma representação textual combinando **descrição, cor, categoria e subcategoria**
- Esta informação é utilizada como **documento** para gerar embeddings e permitir pesquisa semântica
- São também armazenadas **metainformações** relevantes, como nome, preço, link e imagem
- Cada produto recebe um identificador único
- Todos os dados são adicionados à coleção **produtos** no ChromaDB
- No final, é apresentada uma mensagem a confirmar a criação da coleção com sucesso

```
def carregar_produtos_csv():
    limpar_colecao(collection_produtos)

    if not os.path.exists(CSV_FILE):
        print(f"CSV não encontrado: {CSV_FILE}")
        return

    df = pd.read_csv(CSV_FILE)

    documents = []
    metadatas = []
    ids = []

    for i, row in df.iterrows():
        base = str(row.get("Descricao", "")).strip()
        extra = f"{row.get('Cor', '')}{row.get('Categoria_Principal', '')}{row.get('Subcategoria', '')}".strip()
        texto = f"{base} {extra}".strip()

        documents.append(texto)

        metadatas.append({
            "nome": str(row.get("Nome", "")).strip(),
            "preco": str(row.get("Preço", "")).strip(),
            "cor": str(row.get("Cor", "")).strip(),
            "categoria": str(row.get("Categoria_Principal", "")).strip(),
            "subcategoria": str(row.get("Subcategoria", "")).strip(),
            "link": str(row.get("Link", "")).strip(),
            "imagem": str(row.get("Imagem", "")).strip()
        })

        ids.append(str(i))

    collection_produtos.add(
        documents=documents,
        metadatas=metadatas,
        ids=ids
    )

    print("Coleção 'produtos' criada com sucesso!")
```

## ■ App

- Foram desenvolvidas funções para interpretar automaticamente a **query do utilizador**
- A função `extrair_preco_max` identifica valores de preço máximo, permitindo pesquisas como “até 20€”
- A função `extrair_cor` deteta cores mencionadas na query, como azul, preto ou branco
- A função `extrair_tipo` identifica o tipo de produto pretendido (ex: vestido, t-shirt, sapatos)
- Estas funções permitem transformar linguagem natural em **filtros estruturados**
- Este processo melhora a precisão da pesquisa e torna o sistema mais intuitivo para o utilizador

```
def extrair_preco_max(query):
    query = query.lower()

    match = re.search(r"(atè|ate|menos de)\s*(\d+)", query)
    if match:
        return float(match.group(2))

    match = re.search(r"(\d+)\s*€", query)
    if match:
        return float(match.group(1))

    return None

def extrair_cor(query):
    query = query.lower()

    cores = [
        "preto", "preta", "branco", "branca",
        "azul", "azuis", "rosa",
        "vermelho", "vermelha",
        "verde", "verdes",
        "amarelo", "amarela",
        "cinzento", "cinza",
        "bege", "castanho"
    ]

    for cor in cores:
        if cor in query:
            return cor

    return None

def extrair_tipo(query, tipos_disponiveis):
    query = query.lower()

    for tipo in tipos_disponiveis:
        if tipo and tipo in query:
            return tipo

    return None
```



## ■ App

- Foram implementadas funções para identificar automaticamente o **tipo de pergunta do utilizador**
- A função pergunta\_sobre\_documentos deteta questões relacionadas com **informação da loja**, como entregas, devoluções ou a marca
- A função pergunta\_sobre\_produtos identifica perguntas relacionadas com **produtos**, como tipos de roupa, cores ou preços
- Esta classificação permite decidir se a resposta deve usar **dados estruturados (produtos)** ou **dados não estruturados (documentos)**
- Desta forma, o sistema consegue encaminhar a query para a fonte de informação mais adequada, melhorando a qualidade das respostas

```
def pergunta_sobre_documentos(query):  
    query = query.lower()  
  
    palavras = [  
        "marca", "modalfa", "quem é", "história",  
        "entrega", "envio", "devolução",  
        "troca", "reembolso", "prazo"  
    ]  
  
    return any(p in query for p in palavras)  
  
def pergunta_sobre_produtos(query):  
    query = query.lower()  
  
    palavras = [  
        "top", "t-shirt", "vestido", "calça",  
        "saia", "casaco", "sapato", "roupa",  
        "azul", "preto", "branco", "€"  
    ]  
  
    return any(p in query for p in palavras)
```



## ■ App

- Foi implementada uma função para aplicar **filtros aos produtos** com base na query do utilizador
- São considerados três critérios principais: **preço máximo, cor e tipo de produto**
- O preço é convertido para formato numérico, garantindo comparações corretas
- A cor do produto é comparada com a cor identificada na query
- O tipo de produto é verificado através da categoria e subcategoria
- Apenas os produtos que cumprem todos os critérios são incluídos no resultado final
- Este processo permite uma **pesquisa mais precisa e personalizada**, adaptada às necessidades do utilizador

```
def aplicar_filtros(produtos, query):  
    preco_max = extrair_preco_max(query)  
    cor_query = extrair_cor(query)  
  
    tipos = list(set([  
        (p.get("categoria") or "").lower()  
        for p in produtos  
    ] + [  
        (p.get("subcategoria") or "").lower()  
        for p in produtos  
    ]))  
  
    tipo_query = extrair_tipo(query, tipos)  
  
    filtrados = []  
  
    for p in produtos:  
        try:  
            preco = float(str(p.get("preco", "")).replace(",", "."))  
        except:  
            preco = 999999  
  
        cor = (p.get("cor") or "").lower()  
        categoria = (p.get("categoria") or "").lower()  
        subcategoria = (p.get("subcategoria") or "").lower()  
  
        if preco_max and preco > preco_max:  
            continue  
  
        if cor_query and cor_query not in cor:  
            continue  
  
        if tipo_query and not (tipo_query in categoria or tipo_query in subcategoria):  
            continue  
  
        filtrados.append(p)  
  
    return filtrados
```

## ■ App

- Foi implementada uma função para realizar **pesquisa semântica de produtos** utilizando o ChromaDB
- A query do utilizador é utilizada para procurar produtos relevantes na coleção **produtos**
- Os resultados obtidos incluem documentos e metadados associados a cada produto
- A informação relevante (nome, preço, cor, categoria, link e imagem) é extraída dos metadados
- Após a pesquisa, são aplicados **filtros adicionais** com base na query (preço, cor e tipo)
- Os produtos são ordenados por preço, permitindo apresentar as opções mais acessíveis primeiro
- Apenas os **5 melhores resultados** são devolvidos ao utilizador
- Este processo combina **pesquisa semântica com filtragem estruturada**, melhorando a relevância dos resultados

```
def pesquisar_produtos(query):  
    results = collection_produtos.query(query_texts=[query], n_results=20)  
  
    produtos = []  
  
    docs = results.get("documents", [[]])  
    metas = results.get("metadatas", [[]])  
  
    if not docs or not metas or not metas[0]:  
        return []  
  
    for i in range(len(docs[0])):  
        m = metas[0][i]  
  
        produtos.append({  
            "nome": m.get("nome", ""),  
            "preco": m.get("preco", ""),  
            "cor": m.get("cor", ""),  
            "categoria": m.get("categoria", ""),  
            "subcategoria": m.get("subcategoria", ""),  
            "link": m.get("link", ""),  
            "imagem": m.get("imagem", "")  
        })  
  
    produtos = aplicar_filtros(produtos, query)  
  
    return sorted(produtos, key=lambda x: float(str(x["preco"]).replace(".", "")))[-5:]
```

## ■ App

- Foi implementada uma função para realizar **pesquisa semântica em documentos**, utilizando o ChromaDB
- A query do utilizador é usada para obter os **chunks mais relevantes** dos PDFs
- Caso não existam resultados, é devolvida uma lista vazia
- Foram também criadas funções para **construir o contexto** a enviar ao modelo de linguagem:
  - Para produtos, é gerada uma lista com **nome e preço**
  - Para documentos, os textos são concatenados num único bloco
- Este contexto é utilizado pelo modelo para gerar respostas mais **precisas e fundamentadas**
- Este processo é essencial no funcionamento do **RAG**, permitindo combinar recuperação de informação com geração de linguagem natural

```
def pesquisar_documentos(query):  
    results = collection_documentos.query(query_texts=[query], n_results=5)  
  
    docs = results.get("documents", [[]])  
  
    if not docs or not docs[0]:  
        return []  
  
    return docs[0]  
  
def construir_contexto_produtos(produtos):  
    contexto = ""  
    for p in produtos:  
        contexto += f"- {p['nome']} ({p['preco']})€\n"  
    return contexto  
  
def construir_contexto_documentos(documentos):  
    return "\n".join(documentos)
```

## ■ App

- Foi implementada uma função central responsável por **processar a pergunta do utilizador**
- Inicialmente, é feita a classificação da query para determinar se é sobre **produtos ou documentos**
- Caso seja uma pergunta sobre documentos, é realizada uma pesquisa na coleção correspondente e construído o contexto
- Caso seja sobre produtos, é feita a pesquisa na base de dados de produtos e aplicação de filtros
- Se não existirem resultados, é devolvida uma mensagem apropriada ao utilizador
- O contexto obtido é enviado ao modelo de linguagem (**Ollama**) para gerar a resposta final
- A resposta gerada é devolvida juntamente com os produtos (quando aplicável)
- Este processo integra recuperação de informação e geração de texto, constituindo o núcleo do sistema **RAG**

```
def process_query(query):
    usar_docs = pergunta_sobre_documentos(query)
    usar_produtos = pergunta_sobre_produtos(query)

    if usar_docs and not usar_produtos:
        documentos = pesquisar_documentos(query)

        if not documentos:
            return "Não encontrei informação sobre isso.", [], "documentos"

        contexto = construir_contexto_documentos(documentos)

        response = ollama.chat(
            model="qwen2.5:1.5b",
            messages=[{
                "role": "user",
                "content": f"{contexto}\n\nPergunta: {query}"
            }]
        )

        return response["message"]["content"].strip(), [], "documentos"

    produtos = pesquisar_produtos(query)

    if not produtos:
        return "Não encontrei produtos com essas características.", [], "produtos"

    contexto = construir_contexto_produtos(produtos)

    response = ollama.chat(
        model="qwen2.5:1.5b",
        messages=[{
            "role": "user",
            "content": f"{contexto}\n\nPergunta: {query}"
        }]
    )

    return response["message"]["content"].strip(), produtos, "produtos"
```

## ■ App

- Foi criada uma **API REST com Flask** para permitir a comunicação entre o frontend e o sistema
- O endpoint principal (/chat) recebe pedidos do utilizador através do método **POST**
- A mensagem introduzida é extraída do pedido e enviada para a função `process_query`
- Esta função trata toda a lógica de pesquisa e geração de resposta
- A API devolve um objeto em formato **JSON**, contendo:
  - A resposta gerada pelo modelo
  - A lista de produtos encontrados (se aplicável)
  - O tipo de resposta (produtos ou documentos)
- O servidor é executado em modo de desenvolvimento (`debug=True`) para facilitar testes

```
@app.route("/chat", methods=["POST"])
def chat():
    data = request.get_json()
    query = str(data.get("message", "")).strip()

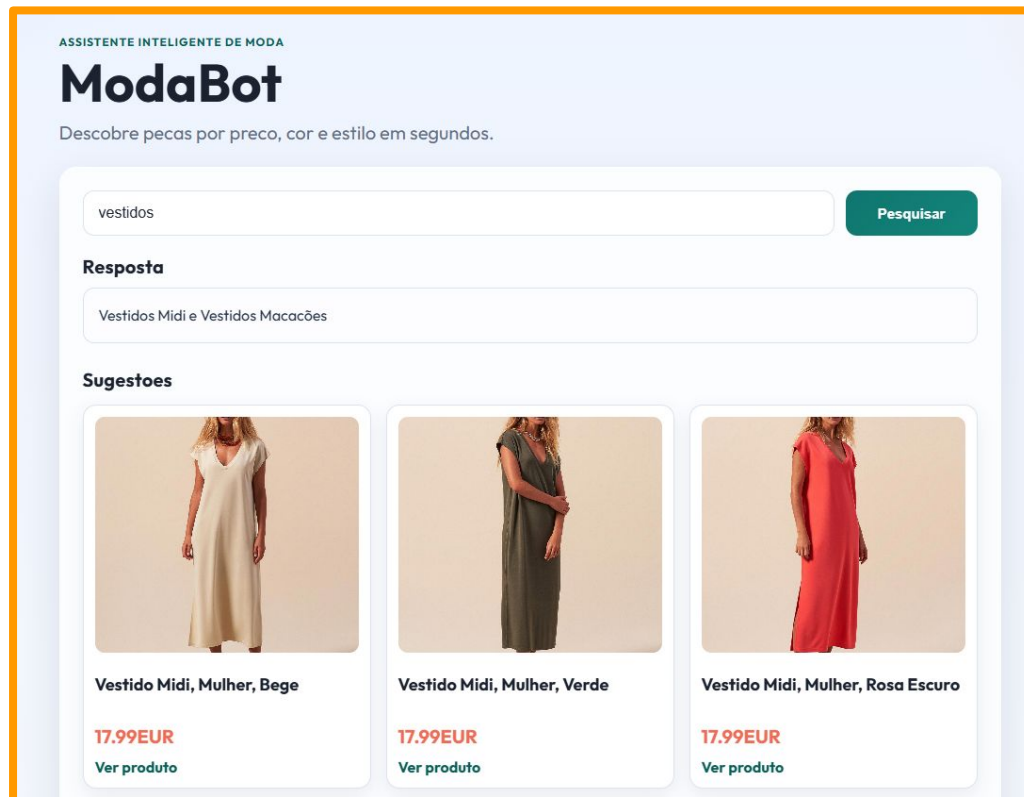
    resposta, produtos, tipo = process_query(query)

    return jsonify({
        "resposta": resposta,
        "produtos": produtos,
        "tipo_resposta": tipo
    })

if __name__ == "__main__":
    app.run(debug=True)
```

## ■ Exemplo de pergunta sobre produtos

- Foi desenvolvida uma **interface web moderna** utilizando **HTML, CSS e JavaScript**
- O utilizador pode introduzir pedidos numa **barra de pesquisa simples e intuitiva**
- A interface comunica com a API Flask através de pedidos **fetch (HTTP POST)**
- Os produtos são exibidos automaticamente em formato de **cards visuais**, contendo: Imagem, Nome, Preço e Link para o produto





## ■ Exemplo de pergunta sobre empresa

- O sistema responde a perguntas sobre **trocas, devoluções e entregas**
- Informação extraída de **documentos PDF da marca (MO/Modalfa)**
- Utilização de **RAG** para recuperar informação relevante
- Permite responder a perguntas fora do catálogo de produtos
- Exemplo: prazo de trocas apresentado diretamente ao utilizador

ASSISTENTE INTELIGENTE DE MODA

# ModaBot

Descobre peças por preço, cor e estilo em segundos.

Pesquisar

**Resposta**

Você poderia fazer uma troca em 30 dias.

## ■ Conclusão

Plataforma de GPTs com RAG usando tecnologias Open Source e dados estruturados e não estruturados

- Concluindo:
  - Foi desenvolvida uma plataforma inteligente de pesquisa de produtos baseada em **RAG**
  - Integra dados **estruturados (CSV)** e **não estruturados (PDF)**
  - Permite responder tanto a pesquisas de produtos como a perguntas sobre a loja
  - Utiliza apenas tecnologias **opensource e locais**
  - Demonstra a aplicação prática de LLMs em contextos reais de e-commerce

## ■ Referências Web

- MO Online. (n.d.). *Quem é a MO*. Disponível em: <https://mo-online.com/pt/sobre-a-mo/quem-e-a-mo/>
- MO Online. (n.d.). *Entregas*. Disponível em: <https://mo-online.com/pt/guia-de-compras/entregas/>
- MO Online. (n.d.). *Devoluções*. Disponível em: <https://mo-online.com/pt/guia-de-compras/devolucoes/>
- MO Online. (n.d.). *Trocas*. Disponível em: <https://mo-online.com/pt/guia-de-compras/trocas/>
- Chroma. (n.d.). *Chroma Documentation*. Disponível em: <https://docs.trychroma.com/>
- Ollama. (n.d.). *Ollama*. Disponível em: <https://ollama.com/>

o teu • de partida



[www.ipvc.pt](http://www.ipvc.pt)